

Resumen Extendido del Curso ELO329: Diseño y Programación Orientados a Objetos

Este documento resume en profundidad los principales temas del curso ELO329, incluyendo explicaciones detalladas y ejemplos relevantes. Se identifican además aquellos contenidos que suelen generar más dificultad y que pueden ser foco de evaluaciones.

1. Fundamentos de Java

Introducción al Lenguaje Java

Java es un lenguaje de programación de propósito general, orientado a objetos, ampliamente utilizado en aplicaciones empresariales, móviles (Android) y sistemas embebidos. Su promesa de "escribir una vez, ejecutar en cualquier lugar" se logra mediante la Java Virtual Machine (JVM), la cual interpreta el bytecode generado por el compilador de Java.

Elementos Básicos

- **Estructura de un programa Java:** todo comienza con una clase pública que contiene el método `main`.
- **Método `main`:** punto de entrada para cualquier aplicación Java ejecutable.
- **Tipos de datos primitivos:** Java tiene ocho tipos primitivos: `int`, `byte`, `short`, `long`, `float`, `double`, `boolean` y `char`.
- **Operadores:** aritméticos, relacionales, lógicos, asignación, incremento/decremento.
- **E/S básica:** lectura con `Scanner`, salida con `System.out.println`.

Temas evaluables frecuentes

- Diferencias entre `==` y `.equals()`.
 - Cuál es el resultado de ciertas expresiones compuestas.
-

2. Programación Orientada a Objetos (OOP)

Clases y Objetos

- Una clase define una estructura de datos y su comportamiento.
- Un objeto es una instancia de una clase.
- Métodos permiten definir acciones que pueden realizar los objetos.

Encapsulamiento

- Protección de los datos internos de una clase.
- Se utilizan modificadores de acceso (`private`, `public`, `protected`).
- Se proveen métodos públicos para acceder (getters) y modificar (setters) los datos internos.

Herencia

- Permite que una clase herede atributos y métodos de otra.
- Se utiliza **extends** para definir una relación de herencia.
- Facilita la reutilización y organización del código.

Polimorfismo

- Capacidad de una referencia de clase padre de representar objetos de sus subclasses.
- Permite definir comportamientos genéricos que cambian según el tipo real del objeto.

Abstracción

- Permite enfocarse en las funcionalidades esenciales.
- Las clases pueden ocultar detalles internos y exponer solo una interfaz pública.

Ejemplo típico de prueba

- Dado un código con clases y herencia, determinar qué métodos se ejecutan en tiempo de ejecución.
-

3. Herencia y Clases Abstractas

Conceptos

- Una clase abstracta es aquella que no puede ser instanciada y puede contener métodos abstractos (sin implementación).
- Útil para definir comportamiento común en una jerarquía de clases.

Uso de **super**

- Se utiliza para acceder a miembros de la clase padre.
- Puede invocar el constructor de la clase padre.

Dynamic Binding

- Asociar métodos a llamadas en tiempo de ejecución (polimorfismo).
- Es el fundamento del comportamiento dinámico en herencia.

Dificultades frecuentes

- Distinguir entre métodos sobrescritos y sobrecargados.
 - Aplicar correctamente **super** y **this** en constructores complejos.
-

4. Interfaces y Colecciones

Interfaces

- Una interfaz define un contrato que puede ser implementado por múltiples clases.
- Permite simular herencia múltiple (una clase puede implementar múltiples interfaces).
- A partir de Java 8, las interfaces pueden tener métodos con implementación (**default**).

Colecciones con ArrayList

- Una estructura de datos dinámica que permite almacenar y acceder a objetos.
- Permite iterar sobre los elementos y realizar operaciones como **add**, **remove**, **contains**, etc.

Casos evaluables

- Implementación de una interfaz para modelar distintos comportamientos.
 - Uso de **ArrayList** para gestionar listas de objetos y aplicar operaciones sobre ellos.
-

5. Clases Internas y Expresiones Lambda

Clases Internas

- Clases definidas dentro de otras clases.
- Se utilizan para encapsular lógicamente funcionalidades auxiliares.
- Tipos: internas estáticas, internas no estáticas, locales, y anónimas.

Método **clone()**

- Permite realizar una copia de un objeto.
- Por defecto realiza copia superficial; para copia profunda se debe sobrescribir el método.

Expresiones Lambda

- Introducidas en Java 8.
- Sintaxis simplificada para implementar interfaces funcionales (una sola función abstracta).
- Se usa con streams, colecciones y en manejo de eventos.

Conceptos clave

- Diferencia entre clase anónima y expresión lambda.
 - Qué es una interfaz funcional y cómo crear una propia.
-

6. Excepciones

Jerarquía

- **Throwable** es la clase raíz.
- Subclases: **Error** y **Exception**.
- Dentro de **Exception**: **IOException**, **SQLException**, etc.

Manejo de excepciones

- Uso de **try-catch-finally**.
- Declaración de métodos con **throws**.
- Generación de excepciones con **throw**.

Errores comunes

- No capturar adecuadamente excepciones que interrumpen el flujo del programa.
 - Confusión entre errores de compilación y errores en tiempo de ejecución.
-

7. Programación Basada en Eventos

Modelo de Eventos

- Un evento es un objeto que representa una acción del usuario.
- Los eventos se manejan mediante `EventHandler` o expresiones lambda.

Controles comunes

- Botones (`Button`), cajas de texto (`TextField`), sliders (`Slider`).

Eventos frecuentes

- `setOnAction`: acción en botones.
- `setOnMouseClicked`: clics de mouse.
- `setOnKeyPressed`: teclas.

Ejercicios comunes

- Diseñar una GUI que reaccione a varios eventos.
 - Implementar lógicas de validación de entrada mediante eventos.
-

8. JavaFX

Componentes

- `Stage`: la ventana principal.
- `Scene`: contiene la jerarquía de nodos.
- `Node`: cualquier objeto visual.

Diseño de Interfaces

- Uso de layouts (`VBox`, `HBox`, `BorderPane`, etc.).
- Estilización con CSS.
- Binding de propiedades para hacer interfaces reactivas.

Animaciones

- `Timeline`: define cuadros de animación.
- `PathTransition`: mueve nodos a lo largo de un camino.
- `FadeTransition`, `RotateTransition`, etc.

Multimedia

- `MediaPlayer` para reproducir audio/video.
- `MediaView` para mostrar videos.

- Ajuste de volumen, pausa/reanudación.

Menús

- `MenuBar`, `Menu`, `MenuItem`.
- `ContextMenu`: menús emergentes.

Evaluación típica

- Dado un código con GUI, identificar el flujo de eventos y la ejecución.
-

9. Paquetes y Documentación

Paquetes

- Organización lógica de clases en directorios.
- Uso de `package` y `import`.
- Resolución de conflictos entre clases con nombres idénticos.

Documentación

- Uso de `javadoc` para generar documentación HTML a partir del código fuente.
- Etiquetas comunes: `@param`, `@return`, `@throws`, `@author`.

Buenas prácticas

- Documentar todas las clases públicas y sus métodos.
 - Mantener una estructura clara de paquetes por módulo funcional.
-

10. Introducción al Desarrollo Android

Arquitectura

- Basado en kernel de Linux.
- Capas: Aplicaciones, Java API Framework, Android Runtime (ART), HAL, Kernel.

Android Studio

- IDE oficial.
- Soporte para emuladores y dispositivos físicos.
- Organización en carpetas: `manifests`, `java`, `res`.

Actividades

- Cada pantalla en Android es una `Activity`.
- Se definen en XML y se enlazan con código Java.

Ejecución

- Se puede probar en un emulador o dispositivo real con `USB debugging`.

Preguntas típicas

- ¿Qué rol cumple el archivo `AndroidManifest.xml`?
- ¿Cómo se estructura el ciclo de vida de una actividad?

Fin del resumen extendido. Se recomienda revisar cada ejemplo práctico provisto en las diapositivas y consolidar estos contenidos mediante implementación de proyectos personales.